DOCKER-COMPOSE

What is Docker-Compose

Docker Compose allows you to run multi-container applications with one command.

For example:

- Frontend container
- Backend container
- Database container

All can be managed together using Docker Compose.

Purpose of Docker Compose

- Create multiple containers
- Connect containers together
- Define ports, volumes, networks
- Build images automatically
- Start and stop complete applications easily

SAMPLE DOCKER-COMPOSE FILE

1.vim docker-compose.yaml

```
version: "3.8"
services:

  webserver:
    image: httpd:latest
    ports:
      - "80:80"
    networks:
      - mynetwork

networks:
  mynetwork:
```

2.docker-compose up -d
3.docker ps
4.<public_ip>

DEPLOY MONOLITHIC APPLICATION USING DOCKER COMPOSE

< Project_url: https://github.com/mayurpandit25/studentapp-ui.git >

1. vim docker-compose.yaml

```
version: "3.8"
services:

  studentapp:
    build: .
    image: studentapp:latest
    container_name: student-app
    ports:
      - "8080:8080"
    restart: always
```

2. docker compose up -d –build
3. docker ps
4. Check it on web
   http://<public_ip>:8080/student

DEPLOY THREE-TIER APPLICATION USING DOCKER COMPOSE

< Project_url: https://github.com/mayurpandit25/EasyCRUD-Updated-df.git >

1. vim docker-compose.yaml

```
version: "3.8"
services:

  backend:
    build:
      context: ./backend
      dockerfile: dockerfile
    ports:
      - "8080:8080"
```

```
  frontend:
    build:
      context: ./frontend
      dockerfile: dockerfile
    ports:
      - "80:80"
    depends_on:
      - backend
```

2. docker compose up -d
3. docker ps
4. docker compose down

Check on web

1. <http://<public_ip>:8080/api>
2. <http://public_ip>

DEPLOY THREE-TIER APPLICATION USING DOCKER-COMPOSE LOCALLY

< Project_url: https://github.com/mayurpandit25/EasyCRUD-Updated-df.git >

Note:

1. change only in application properties –> replace endpoint with name db

1.vim docker-compose.yaml

```
version: "3.8"
services:

  db:
    image: mariadb:latest
    container_name: mysql-db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: student_db
      MYSQL_USER: admin
      MYSQL_PASSWORD: admin123
```

```
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql
    networks:
      - student-network

  backend:
    build:
      context: ./backend
      dockerfile: dockerfile
    container_name: backend
    restart: always
    ports:
      - "8080:8080"
    depends_on:
      - db
    networks:
      - student-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: frontend
    restart: always
    ports:
      - "80:80"
    depends_on:
      - backend
    networks:
      - student-network

volumes:
  mysql-data:
networks:
  student-network:
```

2.docker compose up -d –build
3.docker ps

Access The Application

1. http://localhost:8080/api ---------> backend works
2. http://localhost ----------> frontend works

TOMCAT + REVERSE PROXY

1. vim nginx.conf

```
events {}
http {
      server {
            listen 80;
            location / {
                  proxy_pass http://tomcat:8080;
            }
      }
}
```

2. vim docker-compose.yaml

```
version: "3.8"
services:

  tomcat:
   image: tomcat:10
   container_name: tomcat-container
    ports:
     - "8080:8080"

  nginx:
    image: nginx:latest
    ports:
     - "80:80"
    container_name: nginx-container
    volumes:
     - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
     - tomcat
```

3. docker compose up -d
4. docker ps
5. docker images

6. curl http://localhost:8080 --------> #before reverse proxy
7. curl http://localhost ----------> #after reverse proxy

TOMCAT + NGINX LOAD BALANCING USING DOCKER COMPOSE

1. vim docker-compose.yaml

```
version: "3.8"
services:

  tomcat1:
    image: tomcat:10
    container_name: tomcat-10
    ports:
      - "8081:8080"

  tomcat2:
    image: tomcat:11
    container_name: tomcat-11
    ports:
      - "8082:8080"

  nginx:
    image: nginx:latest
    container_name: nginx-container
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      - tomcat1
      - tomcat2
```

2. vim nginx.conf

```
events {}
http {
      upstream backend {
```

```
        server tomcat1:8080;
        server tomcat2:8080;
    }
    server {
        listen 80;
        location / {
            proxy_pass http://backend;
        }
    }
}
```

3. docker compose up -d
4. docker ps

5. http://localhost:8081
6. http://localhost:8082
7. http://localhost

Nginx distributes requests between both Tomcat containers using Round Robin

LOAD TESTING USING AUTOCANNON ON LOCAL MACHINE

1. apt install nodejs npm -y
2. apt install -g autocannon -y
3. autocannon –version

4. autocannon -c30 -d10 http://localhost:8081
5. autocannon -c30 -d10 http://localhost:8082
6. autocannon -c30 -d10 http://localhost

NGINX down Parameter

```
events {}
http {
    upstream backend {
        server tomcat1:8080;
        server tomcat2:8080;
        server tomcat3:8080 down;
    }
    server {
        listen 80;
        location / {
            proxy_pass http://backend;
```

```
        }
    }
}
```

1. Marks a Server as Unavailable
2. Useful for Testing or Maintenance
3. Easy to Enable or Disable
4. Server Can Still Be Running

NGINX Backup Server Configuration

```
events {}
http {
    upstream backend {
        server tomcat1:8080;
        server tomcat2:8080;
        server tomcat3:8080 backup;
    }
    server {
        listen 80;
        location / {
            proxy_pass http://backend;
        }
    }
}
```

1. backup marks a server as a standby server.
2. It does not receive traffic while at least one primary server is healthy.
3. It starts receiving traffic only when all primary servers are unavailable
4. When a primary server becomes healthy again, NGINX automatically routes traffic back to the primary server.
